

Evaluation Comments Template

General Information

- Evaluator Name: P3
 - Application Name: petclinic
 - Refactoring Approach: monomorph
-

1) Services Boundaries and Decompositions

1. Are all of the classes in the application mapped to their correct microservices based on their decomposition?

Yes, like jpetstore

2. Are there classes that were not included in the decomposition but found in the monolith? How were they handled by the approach?

Describe any missing or unaccounted classes:

No, like jpetstore

3. Are there new classes added to the microservices? Do they still respect the single responsibility of these microservices?

Yes, like jpetstore

4. Has the dependency mapping between components been accurately preserved?

From what I can find, yes

5. How well did the approach handle this aspect? (Score from 1 to 10)

10

6. Additional Comments

2) Correctness & Functional Equivalence:

1. Does the refactored application produce the same outcomes as the original monolith?

Most of the logic seems to remain the same

2. Have any functional behaviors been inadvertently altered during the transformation?

There are some parts I don't think is the same as the monolithic version. For instance, addVisit in the monolithic version is a single line, but in the service version of vets, it checks if getVisits() is null. If it is the case, it adds the visit to a new LinkedHashSet which is not in the monolithic version. I do not understand the implications, but it isn't the same behavior.

3. Is the transformation strictly limited to modifying call mechanics, leaving the business logic unchanged?

Since most original files from the monolithic version aren't modified, most of the of the business logic is left unchanged

4. Do the transformed interactions maintain the same logical flow as in the monolith?

Yes, from what I could see

5. How well did the approach handle this aspect? (Score from 1 to 10)

7

6. Additional Comments

3) Code Transformation and Remote Invocations:

1. Are all relevant local method calls successfully replaced by their remote call counterparts?

Yes

2. Are the request and response structures consistent with the original local calls?

Yes

3. Are there any issues such as missing object mappings or incorrect handling of requests and responses?

Not that I could find

4. Is data correctly serialized and deserialized during remote calls?

Yes

5. How well did the approach handle this aspect? (Score from 1 to 10)

10

6. Additional Comments

4) Data Handling and Consistency:

1. Is there an existing database in the monolith? How did the refactoring approach handle it?

Yes, each service communicates with its own data and to modify a data not related to the service, it calls the other service. Which is what I would expect.

2. How is a shared state managed or synchronized across the services?

It uses gRPC to move objects between services

3. Is there any data duplication? Do you think that it is justified?

I don't see data duplication, but some types are duplicated with the business logic.

4. Is the data still consistent? Were there any new consistency mechanisms added if needed?

Yes, a lease system keeps the data consistent.

5. How well did the approach handle this aspect? (Score from 1 to 10)

8

6. Additional Comments

Data handling and consistency is good, but I wonder if there would be better patterns that would be simpler or better suited to this system. Since it is a small application, there might be a simpler solution even if it incurs a performance overhead.

5) Performance Overhead:

1. Are there new mechanisms introduced that add overhead to existing interactions?

It seems to be using a leasing mechanism to prevent service level race conditions.

2. Do you think that this overhead is justified?

I do not know of alternatives, so it is justified to keep the data consistent. However, I would look for alternative solutions if possible

3. What mechanisms would you replace if you had to modify the code manually?

Just like I said in question 2, I would try to find an alternative to the leasing system.

4. Are there any indications of over-segmentation leading to excessive inter-service communication?

I don't think, but this leasing system might be an indication of over-segmentation. There might be an alternative to it.

5. How well did the approach handle this aspect? (Score from 1 to 10)

8

6. Additional Comments

6) Code Maintainability & Readability Post-Transformation:

1. Is the refactored code easy to understand and follow compared to the original monolith?

Just like jpetstore and 7ep-demo, it is harder than the monolithic version and it introduces abstractions

2. Has the transformation introduced excessive boilerplate code or complexity?

Yes, like jpetstore and 7ep-demo

3. How difficult is it to replace the mechanisms introduced by the approach if needed?

For business simple, but for parts related to microservices, it is harder, like jpetstore and 7ep-demo

4. Does the approach include fault tolerance or error handling mechanisms?

Yes, like jpetstore and 7ep-demo

5. How well did the approach handle this aspect? (Score from 1 to 10)

6

6. Additional Comments

7) Execution and Packaging:

1. Does the refactored version compile and/or package correctly?

No, there were errors about missing functions (verifyInitialized) and some syntax errors such as one file where comments were not formatted appropriately. It also missed an import in VisitController of visits service.

2. Does the refactoring approach introduce new dependencies?

Yes, but only those needed for gRPC

3. Does the approach introduce any new deployment automation mechanisms?

Yes, there are some more initializations steps

4. How well did the approach handle this aspect? (Score from 1 to 10)

6

5. Additional Comments

Most of the application is fine and some of the errors can easily be fixed, but the missing functions are a bit more of an issue.

8) Additional comments that do not fall into any aspect:

Overall, my comments are the same as with jpetstore and 7ep-demo. The leasing system might be necessary in some cases, but I don't think it should be implemented systematically. If not implemented correctly it could lead to hard-to-debug bugs. Also, I find that there is a lot of boilerplate. For complex applications it might be manageable as it will represent a small percentage of the code, but in this case, it almost doubles the amount of code. In the case of small applications, I think this approach wouldn't be suitable, but for big applications, it might be fine.